

NAVEEN AUTOMATION LABS

DRIVEN BY CODE, DEFINED BY QUALITY



MULTIPLE ELEMENTS

all() & allInnerTexts()

Date: _____



Working with Multiple Elements

- When a locator matches **multiple elements** on the page
- Need special methods to **get all matching elements**
- Two main approaches: **all()** and **allInnerTexts()**

! KEY DIFFERENCE!

all() → Returns array of Locator objects (can interact with each)

allInnerTexts() → Returns array of strings (just text content)

Use **all()** when you need to click/interact with elements

Use **allInnerTexts()** when you only need text content



Method 1: allInnerTexts() - Get Text Only

- Returns **array of strings**
- Gets **visible text** from all matching elements
- **Fast & simple** - use when you only need text!

{ } allInnerTexts() Example

```
import { Browser, chromium, Page } from '@playwright/test';

(async () => {
  let browser: Browser = await chromium.launch({
    headless: false,
    channel: 'chrome'
  });

  let page: Page = await browser.newPage();
  await page.goto('https://naveenautomationlabs.com/opencart/');
  // Get all text from matching elements as string array
  let panellinks: string[] = await page
    .locator('div.list-group a')
    .allInnerTexts();
  // Print all texts
  console.log(panellinks);
  // Output: ['Login', 'Register', 'Forgotten Password', ...]
  // Loop through and print each text
  for (let e of panellinks) {
    console.log(e);
  }
  // Find and click specific text
  for (let e of panellinks) {
    if (e === 'Forgotten Password') {
      await page.getByText(e).click();
      break;
    }
  }
  await browser.close();
})();
```

Method 2: all() - Get Locator Objects

- Returns array of Locator objects
- Can interact with each element (click, fill, etc.)
- More flexible - access any property or action!

{ } all() Example - Get Locator Array

```
import { Locator } from '@playwright/test';
// Get array of Locator objects
let panelLocators: Locator[] = await page
.locator('div.list-group a')
.all();
console.log('Total elements:', panelLocators.length);
```

Different Ways to Loop Through all()

{ } Loop Method 1: Traditional for Loop with Index

```
let panelLocators: Locator[] = await page
.locator('div.list-group a')
.all();
// Loop with index
for (let i = 0; i < panelLocators.length; i++) {
  console.log(await panelLocators[i].innerText());
}
```

{ } Loop Method 2: for...of Loop (Recommended!)

```
let panelLocators: Locator[] = await page
.locator('div.list-group a')
.all();
// for...of loop - cleaner!
for (let e of panelLocators) {
  console.log(await e.innerText());
}
```

{ } Loop Method 3: forEach (⚠️ Doesn't Work with Async!)

```
// ❌ This WON'T work correctly! forEach doesn't handle async well
panelLocators.forEach(async e => {
  console.log(await e.innerText());
});
// ❌ Output may be out of order or incomplete!
```

// Don't use `forEach` with `await` - use `for...of` instead!

💡 LOOP RECOMMENDATION:

Use `for...of` loop when working with async operations!

- ✅ `for...of` - Works perfectly with `await`
- ✅ Traditional `for` - Also works, but more verbose
- ❌ `forEach` - Doesn't handle async properly!

Always use `for...of` for cleaner, safer code! 🎯

🔍 Filtering Elements

{ } Filter and Collect Matching Elements

```
let panelLocators: Locator[] = await page
  .locator('div.list-group a')
  .all();
// Filter elements that start with "Forgotten"
let forgot = [];
for (let e of panelLocators) {
  let text = await e.innerText();
  if (text.startsWith('Forgotten')) {
    forgot.push(text);
  }
}
console.log(forgot);
// Output: ['Forgotten Password']
```

🎯 Common Use Cases

{ } Use Case 1: Count Elements

// Count total matching elements

```
let allLinks: Locator[] = await page.locator('a').all();
console.log('Total links on page:', allLinks.length);
```

{ } Use Case 2: Click Specific Element by Text

```
let buttons: Locator[] = await page.locator('button').all();
for (let button of buttons) {
  let text = await button.innerText();
  if (text === 'Submit') {
    await button.click();
    break;
  }
}
```

{ } Use Case 3: Get All Href Attributes

```
let links: Locator[] = await page.locator('a').all();
let allUrls: string[] = [];
for (let link of links) {
  let href = await link.getAttribute('href');
  if (href) {
    allUrls.push(href);
  }
}
console.log('All URLs:', allUrls);
```

{ } Use Case 4: Verify Element Visibility

```
let images: Locator[] = await page.locator('img').all();
let visibleCount = 0;
for (let img of images) {
  if (await img.isVisible()) {
    visibleCount++;
  }
}
console.log(`Visible images: ${visibleCount} out of ${images.length}`);
```

✓ Complete Real-World Example

{ } Full Example with Test

```
import { test, expect } from '@playwright/test';

test('handle multiple elements', async ({ page }) => {
  await page.goto('https://naveenautomationlabs.com/opencart/');

  // Example 1: Get all texts - allInnerTexts()
  let allTexts = await page
    .locator('div.list-group a')
    .allInnerTexts();
  console.log('All link texts:', allTexts);

  // Example 2: Get locators - all()
  let allLocators = await page
    .locator('div.list-group a')
    .all();
  console.log('Total links:', allLocators.length);

  // Example 3: Find and click specific element
  for (let locator of allLocators) {
    let text = await locator.innerText();
    if (text.includes('Register')) {
      await locator.click();
      break;
    }
  }

  // Verify navigation
  await expect(page).toHaveURL(/register/);
});
```

💡 WHEN TO USE WHICH:

Use allInnerTexts() when:

- ✓ You only need text content
- ✓ Not interacting with elements
- ✓ Just checking/printing text values

Use all() when:

- ✓ Need to click/interact with elements

- ✓ Need to get attributes (href, src, class)
- ✓ Need to check visibility/enabled state
- ✓ Need more control over elements

✗ COMMON MISTAKES:

1. Using `forEach` with `async/await`:

✗ `array.forEach(async e => await ...)`

✓ `for (let e of array) { await ... }`

2. Forgetting to `await innerText()`:

✗ `let text = locator.innerText()`

✓ `let text = await locator.innerText()`

3. Not checking array length before looping:

→ Always verify array has elements

4. Using `all()` when `allInnerTexts()` would suffice:

→ Use `allInnerTexts()` for better performance when only text is needed



PRACTICE EXERCISE:

Task: Work with multiple elements

Go to: <https://naveenautomationlabs.com/opencart/>

1. Get all product names using `allInnerTexts()`
2. Print the count of total products
3. Use `all()` to get all product links
4. Find and click on a specific product
5. Get all image src attributes

6. Filter products that contain "MacBook" in their name

Bonus: Get all prices and find the most expensive product!

KEY TAKEAWAYS:

- ✓ `all()` returns array of Locator objects
- ✓ `allInnerTexts()` returns array of strings
- ✓ Use `for...of` loop with `await` (NOT `forEach`!)
- ✓ `all()` is flexible - can interact with elements
- ✓ `allInnerTexts()` is faster when only text is needed
- ✓ Always await async operations in loops
- ✓ Check array length before processing
- ✓ Use `.length` property to count elements

Happy Testing! 🚀

NAVEEN AUTOMATION LABS

DRIVEN BY CODE, DEFINED BY QUALITY